

MODULE 25 · PART 8 - PRACTICE

The Kestrel Capstone

Twenty-four modules of technique. One database, ten deliverables, and a harness that marks you.

PART 8 of 8 MODULE 25 of 26 RUNS about 20 min

What this is not

- ◆ **Not a test with a pass mark.** Nobody is scoring you and nothing is submitted anywhere.
- ◆ **Not a tutorial.** There is no walkthrough, and the brief states the requirement rather than the method.
- ◆ **Not quick.** Ten deliverables against seven million rows is an evening at the very least, and more likely a weekend.
- ◆ **It is the only part of this course you cannot do by watching.** That is the whole design.

THE TAKEAWAY

A capstone that took an hour would be a set of exercises. The length is what makes you build on your own earlier answers, which is where the real difficulty lives.

Watching somebody diagnose a query is not the same as diagnosing one. **Only one of those is a skill.**

What is in the pack

- ◆ **kestrel.duckdb.** Seven million sales lines, nine dimensions, and every planted quirk, generated rather than described.
- ◆ **BRIEF.md.** Ten deliverables, each naming the modules it draws on and the property that will catch you.
- ◆ **verify.py.** The harness. It checks properties, never values, and it runs everything twice.
- ◆ **solution/**, with a worked answer to all ten. Read it when the harness passes, and not before.

THE TAKEAWAY

The generator ships too, so you can rebuild the database, shrink it, or read exactly how each defect was planted once you are finished.

Setup, in three lines

```
1 pip install duckdb
2
3 python3 verify.py --only d01
4     D01    Complete the store-day series
5           skipped (no d01_*.sql in answers)
6
7 # write answers/d01_store_day.sql, then run it again
```

1 One dependency. No warehouse, no account, no cluster, no credentials.

THE TAKEAWAY

The whole point of choosing DuckDB. A capstone that needs a platform is a capstone most people never start.

The shape of an answer

```
1 -- answers/d03_trading_spells.sql
2
3 CREATE OR REPLACE TABLE d03_trading_spells AS
4 WITH traded AS (
5     SELECT store_key, cal_date,
6           ROW_NUMBER() OVER (PARTITION BY store_key
7                               ORDER BY cal_date) AS rn
8     FROM d01_store_day
9     WHERE NOT was_closed
10 )
11 SELECT ...
12 FROM traded ...;
```

3 One file per deliverable, named for the table it builds, ending in a CREATE OR REPLACE TABLE.

8 And it reads d01, because four deliverables do. The harness runs them in order.

THE TAKEAWAY

The file name and the table name have to match, because that is how the harness finds what to check. Everything else about how you write it is yours.

Write it the way this course would

- ◆ **Stack CTEs, one grain per level, and comment the grain.** Module ten, and it is what makes d10 writable.
- ◆ **State every frame explicitly.** Module five. DuckDB will supply RANGE if you do not, exactly like Postgres.
- ◆ **End every ordering that feeds a ROW_NUMBER or a frame with a unique column.** Module two.
- ◆ **None of this is checked.** The harness marks results, not style, and you will feel the absence of all three anyway.

THE TAKEAWAY

The habits are not graded and they are what makes the difference between finishing this in an evening and spending a weekend finding out where a grain changed.

Why DuckDB specifically

- ◆ **It supports every construct this course teaches.** QUALIFY, ROWS, RANGE and GROUPS frames, EXCLUDE, WITH RECURSIVE, percentiles, GROUPING SETS, PIVOT, ASOF JOIN.
- ◆ **Which no other engine in the course does.** Redshift has no RANGE, Postgres 18 has no QUALIFY, SQL Server has neither GROUPS nor EXCLUDE.
- ◆ **One syntax difference to know.** DuckDB writes IGNORE NULLS inside the parentheses: LAG(x IGNORE NULLS) OVER (...).
- ◆ **It is already in the landscape,** as the analysts' local copies of extracts. This costs no fiction.

THE TAKEAWAY

A third syntax position for IGNORE NULLS, after Redshift's and the one coming in Postgres 19. The feature is fifteen years old and nobody has agreed where to put it.

Every defect this course described is
generated into the data. Not described.
Present.

What is actually in there

THE QUIRK	THE MEASURED VALUE
Transactions with lines tied inside one second	30.8%
Sales lines with no promotion	62.0%
Sales lines with no member, so guest shopping	38.0%
Ranged products that have never sold	exactly 1,400
Store-days missing from 63 refurbishment closures	1,266
Categories unreachable because of the cycle	42
Rows in the exclusion list, one with a null key	214

THE TAKEAWAY

These are counted from the shipped file, not asserted. Every percentage this course quoted is a number you can now run a query against.

Which means the failures are real too

- ◆ **A running total with a RANGE frame will plateau**, on roughly a third of transactions, exactly as module five said.
- ◆ **A NOT IN against the exclusion list will return nothing at all**, because one row carries a null. Module two.
- ◆ **A LAG over the store-day series will reach back weeks** wherever a store was closed, unless you completed the series first.
- ◆ **And a dedup ordered by a non-unique column** will pick a survivor you did not choose.

THE TAKEAWAY

None of these will produce an error. All four produce a plausible number, which is the shape this whole course has been about.

Ten deliverables. Between them they need
most of the course.

The ten, and where each comes from

DELIVERABLE	MODULES
d01 Complete the store-day series	6, 11
d02 Category hierarchy, and d02c the cycle	11, 12
d03 Trading spells across the gaps	13
d04 Sessionise the app events	6, 13, 14
d05 Collapse the tier snapshots into spells	13
d06 Price as at the sale	6, 15
d07 The store scorecard	4, 5, 7, 9
d08 Regional summary with subtotals	16, 19
d09 Reconcile, both directions	18, 20
d10 Prove it	2, 5, 20

THE TAKEAWAY

Eleven tables from ten deliverables, because the category one splits in two. Modules 1, 3, 8, 10 and 17 are unlisted, and you will use all five anyway.

Start with d01, and mean it

- ◆ **One row per trading store per day**, with closed days present as rows carrying zero rather than absent.
- ◆ **Four other deliverables read it**, so getting it wrong makes four more things wrong in ways that look unrelated.
- ◆ **It is the fix module six kept pointing at**. Once the series is complete, the previous row is the previous day.
- ◆ **The naive version passes three checks and fails one**. Grouping the facts gives the right totals and the wrong row count.

THE TAKEAWAY

The harness will tell you the row count is wrong and say 'closed days must be present as rows, not absent'. It will not tell you how many rows you should have.

d02 will worry you, and it should not

- ◆ **Your downward walk will not reach all 4,912 categories.** It will reach 4,870, and that is the correct answer.
- ◆ **A node inside a cycle has its parent inside the cycle,** so a walk from the roots can never arrive at it.
- ◆ **Which means it does not hang.** It finishes quickly and silently omits the cycle and everything beneath it.
- ◆ **d02c is the walk that does hang.** Upward, from every node, and that is where the guard earns its place.

THE TAKEAWAY

Two deliverables, one cycle, two opposite failures. The downward walk needs a row-count assertion to catch it. The upward walk needs a path check to survive it.

d03, d04 and d05 are one technique

- ◆ **Trading spells, sessions, and tier spells.** Three different questions, one family of solution.
- ◆ **All three are gaps and islands,** and doing them in order will show you that before anybody tells you.
- ◆ **They differ in what breaks a run.** A missing day, thirty minutes of silence, or a changed value.
- ◆ **And the harness checks the same three properties on each:** full coverage, no overlap, and no run that should have been two.

THE TAKEAWAY

Doing these three consecutively is deliberate. The third one should feel like the first one with a different predicate, and if it does not, go back to the first one.

d04, and the edge that catches everyone

```
1 CASE WHEN event_ts - LAG(event_ts) OVER w > INTERVAL 30 MINUTE
2     THEN 1 ELSE 0 END AS is_new_session
3
4 -- the first event of every member has a NULL gap.
5 -- NULL > INTERVAL 30 MINUTE is UNKNOWN, so this returns 0,
6 -- and the very first session of every member is not flagged.
7 -- 50,000 members, 50,000 missing session boundaries.
```

7 The harness catches it: events will not add up, because the first session of each member merges into nothing.

THE TAKEAWAY

Module two and module six, meeting inside one CASE expression. Every partition has a first row, and here there are fifty thousand of them.

Every partition has a first row

- ◆ **50,000 members in d04. 412 stores in d03. 84,000 products in d06.** Each has an edge, and the edges are not rare.
- ◆ **A member with one event is entirely edge.** Previous and next are both null and there is no middle at all.
- ◆ **Decide what the first row means, deliberately,** for each deliverable. Usually it is a session, a spell, or a change.
- ◆ **The harness checks coverage, which is exactly this.** Events must add up, spells must tile, months must all be covered.

THE TAKEAWAY

Module six said handle the edge as the common case rather than as an exception. On this data the edge is between one and five percent of every result.

d06 is where the grain goes

- ◆ **An interval join that fans out is the failure here**, and it does not error. It adds rows.
- ◆ **Exactly one row per sales line, seven million of them.** The harness counts, and it also checks sale keys are unique.
- ◆ **And it checks you found the nearest earlier price**, not just an earlier one. Those are different queries.
- ◆ **DuckDB has ASOF JOIN**, which does this directly. Solving it without one is worth doing at least once.

THE TAKEAWAY

The second check is the interesting one. A query that matches an earlier price rather than the nearest earlier price returns exactly the right number of rows and the wrong values.

d07 has the check that matters most

- ◆ **Share of region, times region share of nation, equals share of nation.** The harness asserts it to six decimal places.
- ◆ **Shares summing to one will not catch a filtered denominator,** because a filtered denominator also sums to one.
- ◆ **The two levels multiplying out will catch it,** because the levels then disagree with each other.
- ◆ **This is module seven's scenario one,** as an assertion rather than as a story.

THE TAKEAWAY

The strongest check in the whole harness, and the one worth taking away. Any nested share has this property, and almost nobody tests it.

d08 and the flag that matters

```
1 SELECT COALESCE(region_key::VARCHAR, 'ALL')    AS region,  
2         COALESCE(store_format, 'ALL FORMATS')  AS store_format,  
3         region_key IS NULL AS is_national_total,  
4         store_format IS NULL AS is_region_subtotal,  
5         net_sales  
6 FROM m;
```

4 Computed before the COALESCE would hide it. After coalescing, a subtotal null and a real null look identical.

THE TAKEAWAY

GROUPING SETS produces nulls that mean 'all values', and your data may contain nulls that mean 'unknown'. If you cannot tell them apart, neither can the consumer.

The subtotal that is not a sum of rows

- ◆ **The grand total is not the sum of every row in the result.** Adding subtotals to detail rows double counts everything.
- ◆ **It is the sum of the region subtotals,** or equivalently the sum of the detail rows. Never both together.
- ◆ **The harness asserts both relationships,** so a result where they disagree fails even though every number in it is real.
- ◆ **This is the hierarchy problem from module nine,** arriving as a pivot rather than as a tree.

THE TAKEAWAY

Every dashboard that totals a column containing subtotals gets this wrong once. The flag is what lets a consumer filter them out before summing.

d09, and why both directions

```
1 SELECT 'net_sales' AS measure,  
2       (SELECT ROUND(SUM(net_amount),2) FROM fact_sales_line) AS from_fact,  
3       (SELECT ROUND(SUM(net_sales), 2) FROM d01_store_day) AS from_model,  
4       (SELECT ROUND(SUM(net_sales), 2) FROM d07_store_scorecard) AS from_scorecard  
5 UNION ALL  
6 SELECT 'baskets',  
7       (SELECT COUNT(*) FROM (SELECT DISTINCT store_key, date_key, txn_id  
8                               FROM fact_sales_line)),  
9       (SELECT SUM(baskets) FROM d01_store_day),  
10      (SELECT SUM(baskets) FROM d07_store_scorecard);
```

7 A basket is store, day and txn_id together. txn_id alone is not unique, which module two established.

THE TAKEAWAY

Line 7 is the deliverable inside the deliverable. Counting DISTINCT txn_id gives you a number that looks reasonable and is wrong by a factor of several hundred.

A money measure and a count measure

- ◆ **They fail differently, which is why the brief asks for both.** A sum survives a fan-out by growing.
- ◆ **A count can survive it without changing,** if you used DISTINCT to hide the duplication rather than fixing it.
- ◆ **So a reconciliation on money alone can pass on a broken grain,** and a count alone can pass on a broken measure.
- ◆ **Two measures, three sources, six numbers.** All six must agree, and the harness checks every pair.

THE TAKEAWAY

This is module twenty's argument in miniature. One assertion that passes proves very little; the value is in choosing assertions that fail differently.

d10 is **your own assertions**. At least six, and every one must hold.

An assertion worth stealing

```
1 SELECT 'd07 nested shares multiply out' AS assertion,  
2     NOT EXISTS (  
3         SELECT 1 FROM d07_store_scorecard  
4             WHERE ABS(share_of_region * region_share_of_nation  
5                     - share_of_nation) > 0.000001  
6     ) AS holds
```

5 Six decimal places, because a filtered denominator is usually wrong by a factor, not by a rounding error.

THE TAKEAWAY

Take this one to work. Any report with a share at two levels has this property, and checking it costs one row and catches the defect that sum-to-one cannot.

Six assertions to start from

- ◆ **Grain.** Row count equals distinct count of the declared key, at every level you built.
- ◆ **Coverage.** Spells cover every day, sessions cover every event, spells cover every month.
- ◆ **Reconciliation.** The measure total is unchanged between the layers where it should be.
- ◆ **Shares.** They sum to one within each group, and the two levels multiply out.
- ◆ **Bounds.** No percentile outside its minimum and maximum, no negative count, no date after today.
- ◆ **Nothing hit a cap.** No recursive walk reached its depth limit, which would mean it was truncated.

THE TAKEAWAY

Six is the minimum the harness accepts and it is also roughly the right number. Any one of them failing tells you something different from the other five.

The harness checks **properties, not values**.
It never tells you the answer.

What a failure looks like

```
1 D01 Complete the store-day series
2 pass one row per store per day
3 FAIL every trading store, every day in the window
4 closed days must be present as rows, not absent
5 pass closed days carry zero, not null
6 pass net sales tie back to the fact table
7 pass identical on a second run
8
9 4/5 checks passed
```

4 The hint names the property. It does not say how many rows you should have, or how to build the spine.

THE TAKEAWAY

Three of the four content checks passed on a query that had grouped the facts. The totals reconciled perfectly. Only the row count knew.

The hints, and what they refuse to say

- ◆ **They name the property that failed.** 'A share's denominator must cover the whole region.'
- ◆ **They never give a number you have to work out.** No expected row counts, no expected totals.
- ◆ **They point at the module when the module is the answer.** 'Think about why some categories are unreachable.'
- ◆ **Which means a failing check is a hypothesis,** not an instruction. You still have to work out what to change.

THE TAKEAWAY

If the harness told you the expected row count you would fix the count. Naming the property makes you fix the query, which is a different thing.

And it runs everything twice

```
1      FAIL the second run differed (1,284 rows)
2          an ORDER BY somewhere does not uniquely
3          identify a row
4
5 # every deliverable is executed a second time into a
6 # separate table, and the two results are compared with
7 # EXCEPT in both directions.
```

6 EXCEPT both ways, because a one-directional comparison misses a row that moved rather than vanished.

THE TAKEAWAY

A deliverable that passes every content check and differs between runs fails. A query with a set of permitted answers does not have an answer.

Passing the determinism check is **a backstop, not a proof.**

So the guarantee still comes from you

- ◆ **A single-node engine with a stable plan will repeat itself**, even when the query permits several answers.
- ◆ **The same query on a distributed warehouse may not**, which is where module two's fraud extract came from.
- ◆ **So read your own ORDER BY clauses**. Does the last column uniquely identify a row, in every one of them?
- ◆ **The test agreeing with itself twice is evidence**. The total order is the guarantee.

THE TAKEAWAY

A harness that claimed to prove determinism would be teaching you the wrong lesson. It can only fail you, never clear you, and the brief says so.

How to work through it

- ◆ **d01 first, then d02 and d02c**, because four deliverables read d01 and you want it settled.
- ◆ **Then d03, d04 and d05 together**, in that order, because they are one technique three times.
- ◆ **d06, d07 and d08 in any order**. They are independent, and d07 is the most satisfying.
- ◆ **d09 and d10 last**, because they are about the other nine and cannot be written before them.

THE TAKEAWAY

Run the harness after every deliverable rather than at the end. A failure in d07 caused by d01 is a great deal easier to find if d01 was green an hour ago.

When you are stuck

- ◆ **Select from your intermediate blocks.** Module ten's debugging property: comment out the final SELECT and look at each level.
- ◆ **Count rows at every grain,** and find the level where the count stopped being what you expected.
- ◆ **Build a five-row fixture** and run your logic against that. Every technique in this course is testable on five rows.
- ◆ **Do not open the solution.** It will make sense immediately and teach you nothing, which is the trap.

THE TAKEAWAY

The fixture is the most underused of these. Seven million rows tells you that something is wrong. Five rows tells you what.

When the harness goes green

THEN READ THE SOLUTION

Compare, do not copy

- Yours passed, so yours is correct
- Look for where it is shorter
- Look for what it asserted that you did not
- Differences are usually taste

DO NOT READ IT FIRST

It costs you the exercise

- Every answer looks obvious once seen
- Recognition is not recall
- You lose the only cold retrieval you get
- And you will believe you knew it

THE TIEBREAKER

The solution is not the only correct answer to any deliverable. Where yours differs and still passes, yours is as good, and it is worth working out why both work.

What finishing this proves

- ◆ That you can complete a **series** before computing anything that depends on adjacency.
- ◆ That you write **total orders**, because four of the ten deliverables quietly depend on it.
- ◆ That you check a **denominator's population** rather than checking that shares sum to one.
- ◆ And that you **assert grain**, at every level, because d10 made you write it down.

THE TAKEAWAY

Four habits, not ten techniques. The techniques are lookupable and the habits are what stop a plausible wrong number reaching a report.

Roughly how long it takes

- ◆ **d01 and d02 in an hour or so**, and d02 will take longer than you expect because of what it does not return.
- ◆ **d03, d04 and d05 faster than that combined**, once the first one has taught you the shape.
- ◆ **d06 and d07 are the substantial two**. Most people spend the largest share of their time on d07.
- ◆ **d09 and d10 in under an hour**, and they are the ones that find the mistakes in everything above them.

THE TAKEAWAY

If d09 and d10 take you a long time, that is the capstone working. They are cheap to write and expensive to satisfy, and what they find is the point.

The techniques are lookupable. **The habits are what you are practising.**

This module has **no worked scenario**. The scenario is a hundred and seventy megabytes.